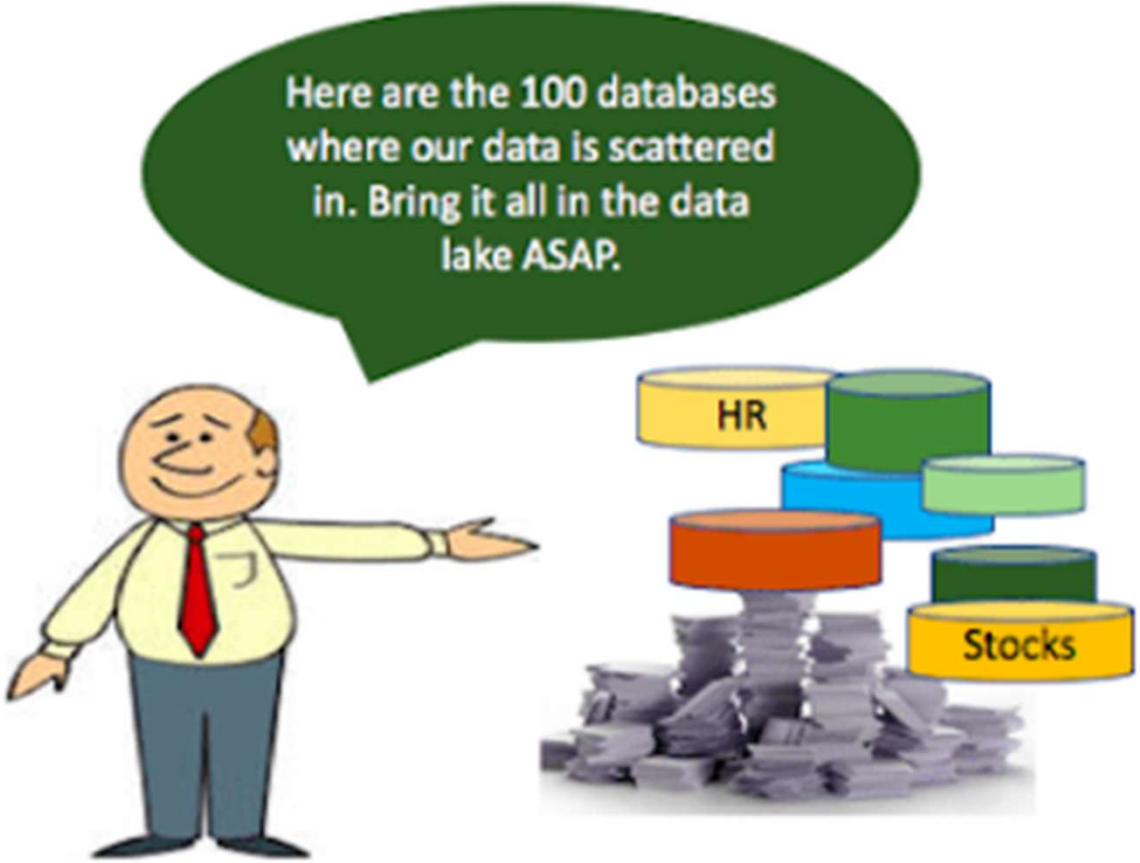
CH
Scoop

- Introduction : data ingestion
- Sqoop
  - Presentation
  - Usage
  - Architecture
  - Data Transfer
- Import / export
- Annex

https://sqoop.apache.org/docs/1.99.7/index.html

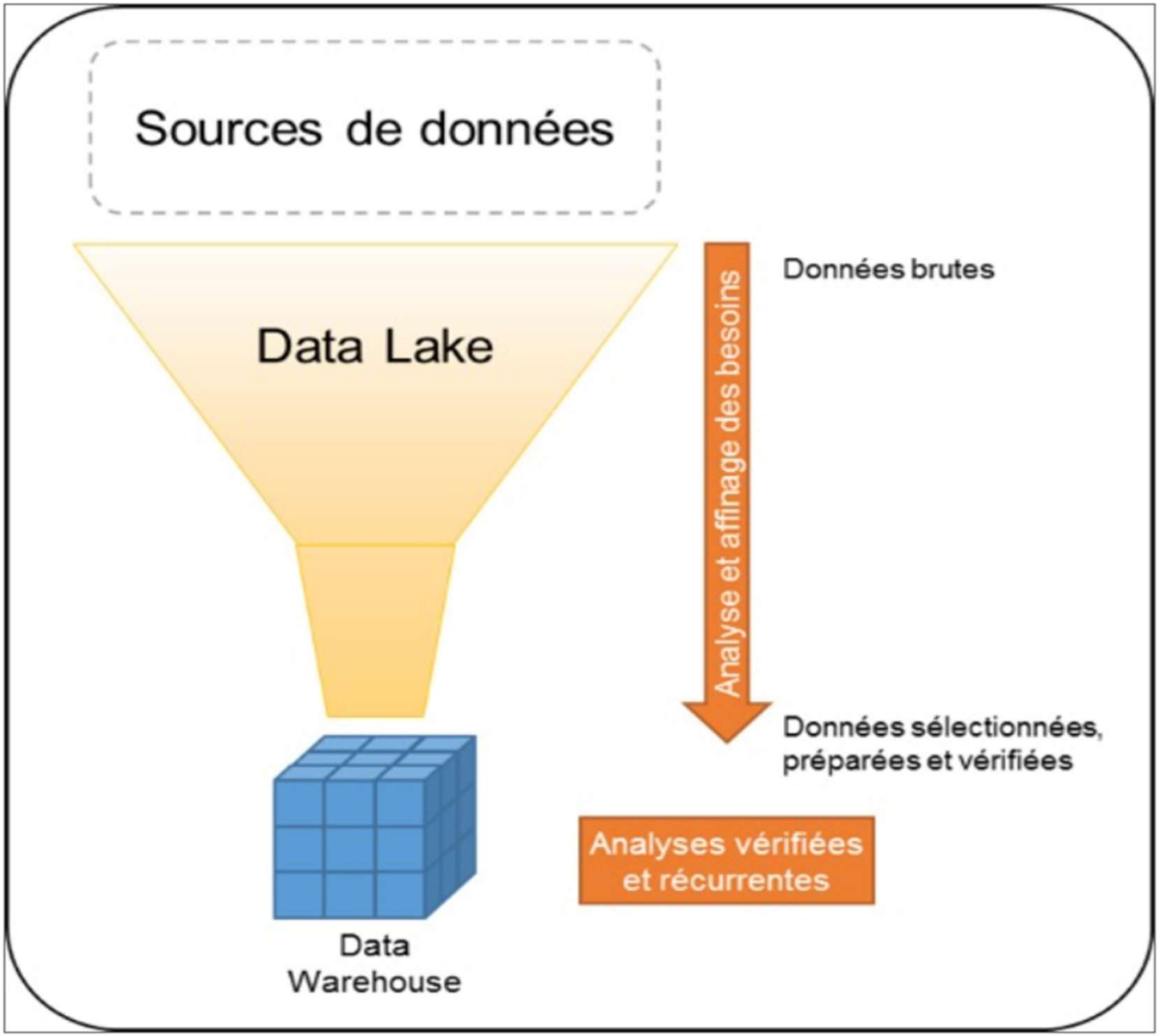
- **Data ingestion to Hadoop !**

Hadoop Data ingestion is the beginning of data pipeline in a data lake. It means taking data from various silo databases and files and putting it into Hadoop.



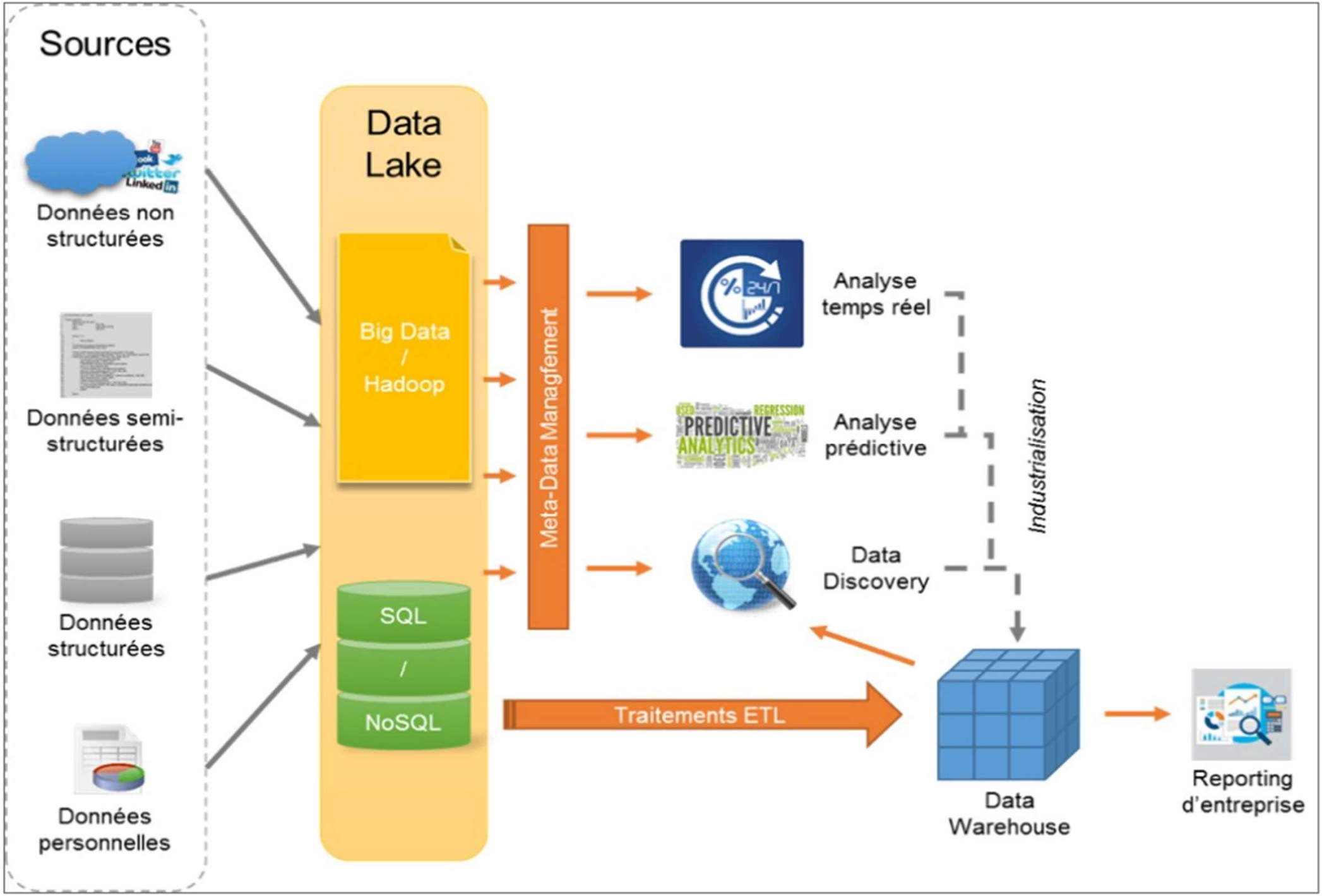
- **The Data Lake**
  - Data Lake is a centralized repository system that enables large volumes of structured, semi-structured and unstructured data to be stored and managed at low cost, in their raw form as **they arrive**. The data stored is not intended to be modelled.
  - The underlying idea of a Data Lake architecture is to have a single space that collects and archives all the data required by the company. This may be raw data, copied from source systems or external sources. But it can also be transformed data that can be used for a variety of tasks, including reporting, analysis or machine learning.
  - Data Lake is a "**schema on read**" architecture. This means that the data stored in the Data Lake is structured and interpreted as it is read, by the user seeking to exploit the data. This offers greater flexibility when exploiting the data, without being constrained by a predetermined schema. But it also requires a high level of control over the data being manipulated. In contrast, the Data Warehouse is of the "**schema on write**" type, with the data being structured at the time of storage so that it can be used directly.

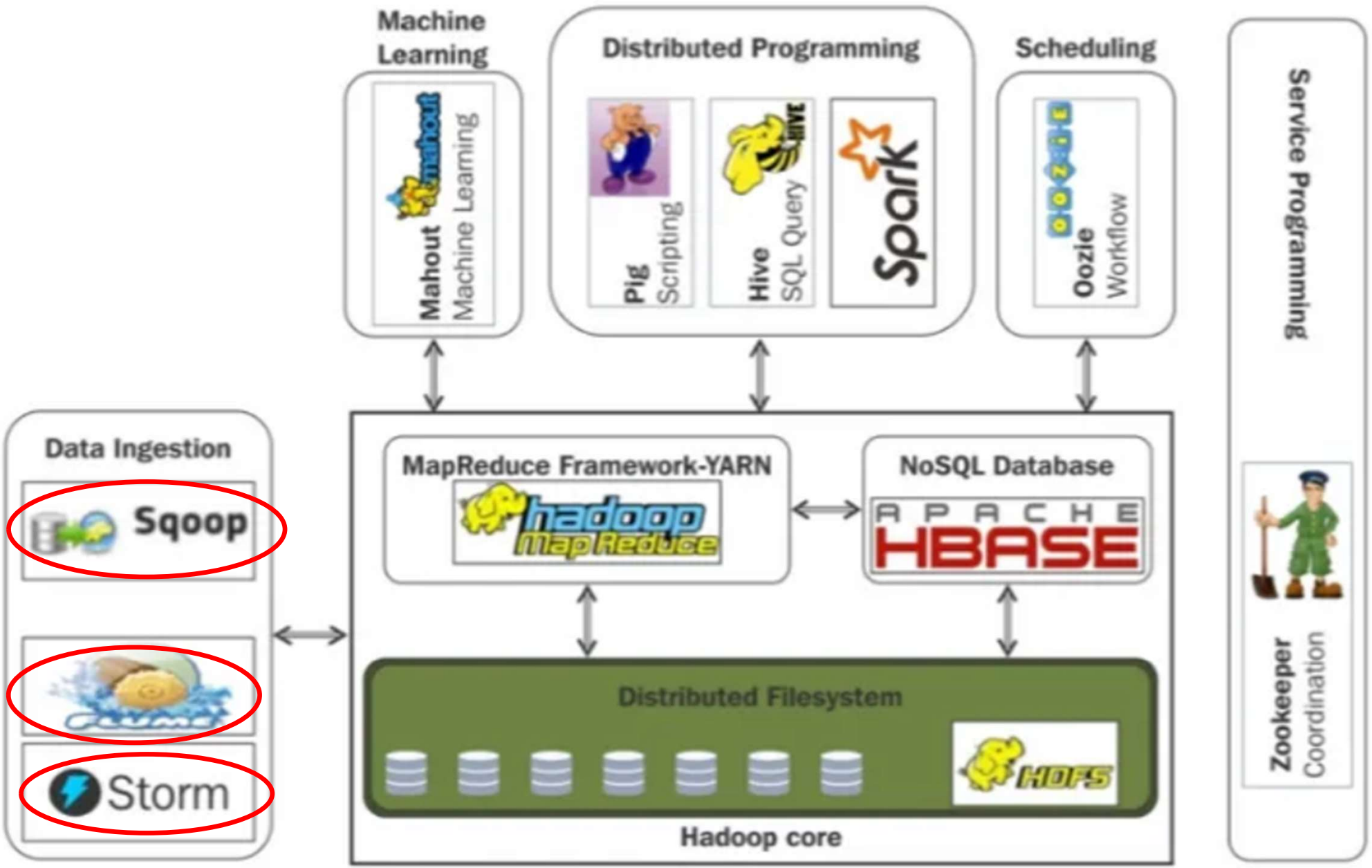
- Data Warehouse System
  - A Data Warehouse is a system that stores and organizes data from various sources to facilitate analysis and decision-making.
  - It enables heterogeneous data to be **combined, cleansed, integrated and structured** according to relevant dimensions and measures.
  - It can be hosted on a server in a Data Centre or in the Cloud, depending on performance, security and scalability requirements.
  - It can be **used to power advanced analysis tools** such as data mining, artificial intelligence (AI) or machine learning (ML).
  - It gives users quick and easy access to strategic information such as trends, patterns, correlations and predictions.


Sources de données
Data Lake
Analyse et affinage des besoins
Données brutes
Données sélectionnées, préparées et vérifiées
Analyses vérifiées et récurrentes
Data Warehouse

Introduction

Data Lake Vs Data Warehouse




Machine Learning
Mahout Machine Learning
Distributed Programming
Pig Scripting
Hive SQL Query
Spark
Scheduling
Oozie Workflow
Service Programming
Zookeeper Coordination
Data Ingestion
Sqoop
Flume
Storm
MapReduce Framework-YARN
hadoop MapReduce
NoSQL Database
APACHE HBASE
Distributed Filesystem
HDFS
Hadoop core

- **Data ingestion** is a crucial process in the field of data engineering, as it involves extracting data from various sources and loading it into a data warehouse or data lake for further analysis and processing.

- One popular tool used for data ingestion is **Apache Sqoop**. Scoop, short for Apache Sqoop, is an open-source framework that enables efficient transfer of data between Apache Hadoop and structured data stores such as relational databases.

- Sqoop, short for SQL-to-Hadoop, is a utility for transferring data from a relational database to HDFS and from HDFS to relational databases.

- For the purpose of importing and exporting massive amounts of data from databases into Hadoop and vice versa, Scoop offers a straightforward and scalable solution. Its extensive database compatibility, which includes Oracle, MySQL, PostgreSQL, and SQL Server, makes it an adaptable data input solution. Scoop makes it simple to send data in parallel, which speeds up data intake and processing.

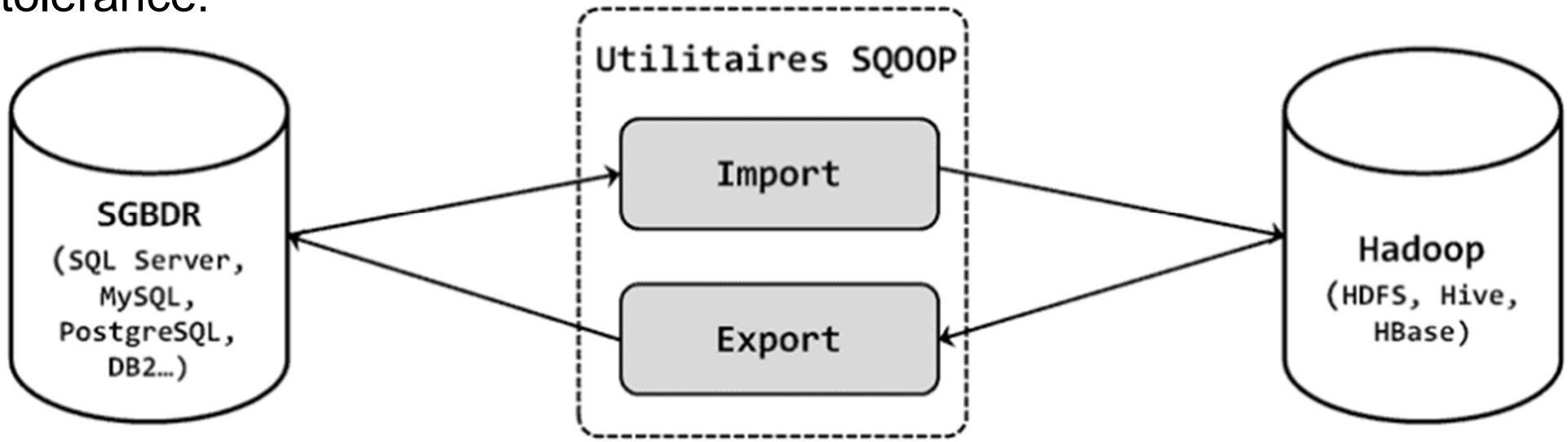
- The main advantage of using Scoop for data ingestion is its ability to handle large datasets efficiently. Scoop may send data in parallel by utilizing parallel processing, which drastically cuts down on the amount of time needed for intake. Additionally, Scoop reduces the possibility of mistakes by automatically generating the code required to import or export data, doing away with the necessity for manual programming. This makes it an easy-to-use data tool.

- Example: Company A has a large customer database stored in an Oracle database. They want to perform analysis on this data using Hadoop. They can use Scoop to easily transfer the customer data from the Oracle database to the Hadoop cluster. Scoop will handle the parallel processing and generate the necessary code, making the data ingestion process fast and error-free.

- Example: Company B receives large amounts of sensor data from their IoT devices, which is stored in a SQL Server database. They want to transfer this data to their data lake in Hadoop for further analysis. Scoop can be used to efficiently extract the sensor data from the SQL Server database and load it into the data lake. The parallel processing capability of Scoop ensures fast ingestion and the automatic code generation reduces the chances of errors.

- Sqoop is a tool designed to transfer data between Hadoop and relational databases.
- Sqoop is used to import data from a relational database management system (RDBMS) such as MySQL or Oracle or a mainframe into the Hadoop Distributed File System (HDFS), transform the data in Hadoop MapReduce, and then export the data back into an RDBMS.

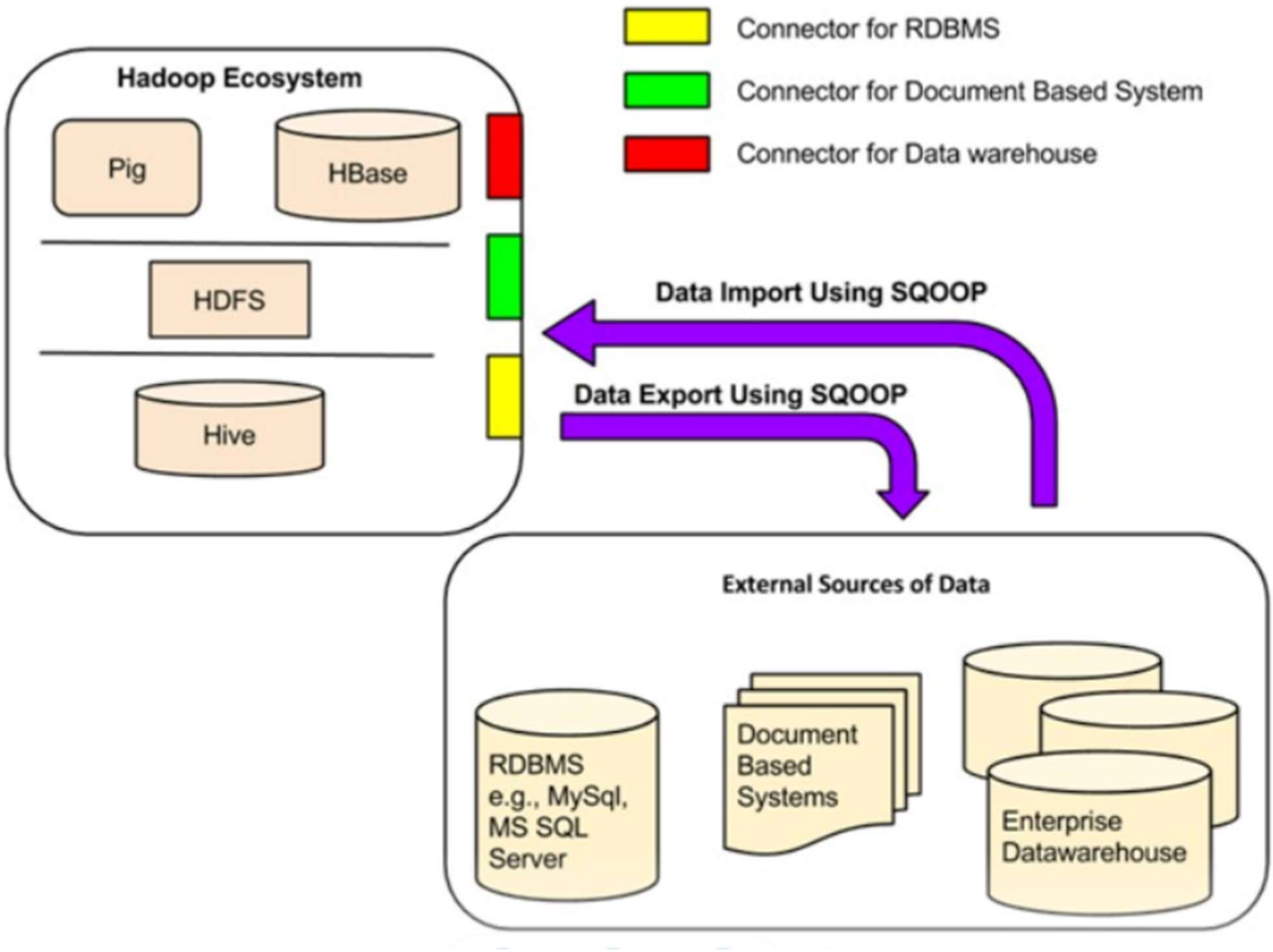
- Sqoop automates most of this process, relying on the database to describe the schema for the data to be imported. Sqoop **uses MapReduce** to import and export the data, which provides parallel operation as well as fault tolerance.



- **What Sqoop Imports:**
  - Data sources: Sqoop can import data from two main sources:
    - ✓ Relational database systems: This includes common databases like MySQL, Oracle, and PostgreSQL.
    - ✓ Mainframe datasets: This refers to structured data stored on legacy mainframe systems.
- **Import Process:**
  - Reading data:
    - ✓ Databases: Sqoop reads data from tables in the database row-by-row.
    - ✓ Mainframes: Sqoop reads records from each mainframe dataset.
  - Output: The imported data is stored in HDFS as a set of files.
  - Parallelism: Sqoop performs the import process in parallel, which means it can read and write data simultaneously to improve speed.
  - File formats: The output files can be in various formats:
    - ✓ Delimited text: Data is separated by characters like commas or tabs (e.g., CSV).
    - ✓ Binary formats: Avro and SequenceFiles store serialized record data, offering efficiency and richer data types.

- Notes:
  - Sqoop offers flexibility in configuring your import job, including specifying which columns to import, filtering data, and customizing file formats.
  - Sqoop integrates with other Hadoop ecosystem tools like Hive and Pig, allowing you to further process and analyze the imported data.

Scoop
Architecture



- Sqoop makes data movement between Hadoop and other storage systems easier by using its connectors.
- Sqoop can interface with several popular relational databases, including MySQL, PostgreSQL, Oracle, SQL Server, and DB2, thanks to these connectors.
- Every connection initiates communication with the matching DBMS that it is linked to.
- Sqoop may connect to any database that complies with the JDBC standard using a generic JDBC connector.
- Sqoop Big Data also provides PostgreSQL and MySQL-optimized connectors that use database-specific APIs to improve speed.
- Sqoop for big data may be used with a range of third-party data storage connectors, including Netezza, Teradata, and Oracle (including Couchbase) business data warehouses and NoSQL stores. These connectors are not part of the basic Sqoop package; instead, users must download them individually and incorporate them into an already-existing Sqoop installation in order to utilize them.

- **1) Partitioning the Dataset**

Sqoop doesn't transfer the entire dataset at once. Instead, it intelligently divides the data into smaller, manageable chunks called partitions. This partitioning strategy offers several benefits:

- Parallel processing: Each partition can be processed simultaneously by different mappers in a MapReduce job, significantly speeding up the transfer process.
- Fault tolerance: If one mapper encounters an error during processing, only that specific partition is affected, not the entire transfer.
- Scalability: Sqoop can scale efficiently to handle very large datasets by increasing the number of partitions and mappers.

- **2) Transferring Individual Partitions**

For each partition, Sqoop creates a distinct mapper in a MapReduce job. These mappers act as independent workers, responsible for:

- Fetching data: The mapper connects to the database using the appropriate connector and retrieves the data from the assigned partition.
- Data conversion: Sqoop utilizes the database information to understand the data types (e.g., integer, string). It then converts each data record into a format suitable for Hadoop, ensuring type-safety and data integrity.
- Storing the data: Finally, the mapper writes the converted data to HDFS (typically as text files, Avro, or SequenceFiles).

- **3) Ensuring Data Integrity**

Sqoop doesn't simply move raw data bytes. It controls the database schema information to understand the data types of each column. This allows for:

- Accurate conversion: Data records are converted to their corresponding Hadoop data types
- Error detection: Inconsistent data or incompatible types are identified and flagged during the transfer, preventing incorrect data from entering HDFS.

- **Sqoop Import :**

The procedure is carried out with the aid of the sqoop import command. We can import a table from the Relational database management system to the Hadoop database server with the aid of the import command. Each record loaded into the Hadoop database server as a single record is kept in text files as part of the Hadoop framework. While importing data, we may also load and split Hive. Sqoop also enables the incremental import of data, which means that if we have already imported a database and want to add a few more rows, we can only do it with the aid of these functions, not the entire database.

See demonstration in the class

Sqoop Export:

The Sqoop export command facilitates the execution of the task with the aid of the export command, which performs operations in reverse. Here, we can transfer data from the Hadoop database file system to the relational database management system with the aid of the export command. Before the operation is finished, the data that will be exported is converted into records. Two processes are involved in the export of data: the first is searching the database for metadata, and the second is moving the data.

See demonstration in the class

Import / export

Incremental

- incremental mode refers to a feature that allows sqoop to import only new or updated data from a relational database into Hadoop, rather than importing the entire dataset every time. This is particularly useful for large datasets or frequently changing data, as it significantly reduces the amount of data transferred and processed, saving time and resources.
- Sqoop offers two main types of incremental imports:

1.Append Mode:

- ✓ Best suited for tables where new rows are continuously added with increasing row ID values.
- ✓ You specify a check column that contains unique row IDs (e.g., an auto-incrementing primary key).
- ✓ Sqoop imports only rows with a check column value greater than the specified last-value (stored from a previous import).

2.Last-Modified Mode:

- ✓ Ideal for tables where existing rows might be updated, and these updates need to be reflected in your Hadoop data.
- ✓ Requires a last-modified column in the database table that tracks the last update timestamp for each row.
- ✓ Sqoop imports only rows with a last-modified value greater than the specified last-value.

Annex : File storage

- Organizations typically store data in structured, semi-structured or unstructured formats to record details of entities (for example, customers and products), specific events (such as business transactions) or other information in documents, images and other formats. The stored data can then be retrieved for analysis and reporting at a later date.

- There are two main categories of commonly used data stores:
  - **File stores**
  - **Databases**

- Text vs. Binary Formats:
  - **Text-based formats** are easier to use and can be read and modified by end users using a simple text editor. These formats are often compressed to reduce storage space.
  - **Binary formats**, on the other hand, require specialized tools or libraries for creation and consumption. However, they offer better performance by optimizing data serialization.

- The specific file format used to store data depends on a number of factors, including:
  - The type of data stored (structured, semi-structured or unstructured).
  - The applications and services that will need to read, write and process the data.
  - The need for data files to be human-readable, or optimized for efficient storage and processing.

- **Delimited text files**

Data is often stored in plain text format with specific field delimiters and line end indicators. The most common format for delimited data is comma-separated values (CSV) where fields are separated by commas, while lines end with a carriage return/new line. Delimited text is a good choice for structured data that needs to be accessible by a wide range of applications and services in a human-readable format.

- Ex.

```
FirstName,LastName,Email
Joe,Jones,joe@litware.com
Samir,Nadoy,samir@northwind.com
```

- **JSON (JavaScript Object Notation)**

JSON is a ubiquitous format in which a hierarchical document schema is used to define data entities (objects) that have multiple attributes. Each attribute can be an object (or a collection of objects), making JSON a flexible format suitable for structured and semi-structured data.

```
{
  "customers":
  [
    {
      "firstName": "Joe",
      "lastName": "Jones",
      "contact":
      [
        {
          "type": "home",
          "number": "555 123-1234"
        },
        {
          "type": "email",
          "address": "joe@litware.com"
        }
      ]
    },
```

```
{
      "firstName": "Samir",
      "lastName": "Nadoy",
      "contact":
      [
        {
          "type": "email",
          "address": "samir@northwind.com"
        }
      ]
    }
  ]
}
```

XML (Extensible Markup Language)

XML est un format de données lisible par l'homme qui était populaire dans les années 90 et 2000. Il a été largement remplacé par le format JSON moins bavard, mais il existe toujours des systèmes qui utilisent XML pour représenter les données. XML utilise des balises placées entre crochets (../) pour définir des éléments et des attributs, comme montré dans cet exemple :

```
<Customers>
  <Customer name="Joe" lastName="Jones">
    <ContactDetails>
      <Contact type="home" number="555 123-1234"/>
      <Contact type="email" address="joe@litware.com"/>
    </ContactDetails>
  </Customer>
  <Customer name="Samir" lastName="Nadoy">
    <ContactDetails>
      <Contact type="email" address="samir@northwind.com"/>
    </ContactDetails>
  </Customer>
</Customers>
```

BLOB (Binary Large Object)

- Ultimately, all files are stored as binary data (1s and 0s), but in the human-readable formats, the bytes of binary data are mapped to printable characters (usually via a character encoding scheme such as ASCII or Unicode). However, some file formats, particularly for unstructured data, store data as raw binary data that must be **interpreted by applications** and rendered. Common types of data stored as binary data include images, video, audio and application-specific documents.
- When working with data of this type, data professionals often refer to data files as **BLOBs** (Binary Large Objects).

- While human-readable formats for structured and semi-structured data can be useful, they are generally not optimized for storage space or processing. Over time, specialized file formats that allow compression, indexing and efficient storage and processing have been developed.

Optimized file formats

- **Avro** is a line-based format. It was created by Apache. Each record contains a header that describes the data structure of the record. This header is stored in JSON format. The data is stored as binary information. An application uses the header information to analyze the binary data and extract the fields it contains. Avro is a good format for compressing data, reducing storage and network bandwidth requirements to a minimum.

- **ORC** (Optimized Row Columnar format) organizes data in columns rather than rows. It was developed by HortonWorks to optimize read and write operations in Apache Hive (Hive is a data warehouse system that supports fast data aggregation and querying on large datasets). An ORC file contains bands of data. Each strip contains the data for one column or a set of columns. A strip contains an index into the rows of the strip, the data for each row and a footer containing statistical information (number of rows, sum, max, min, etc.) for each column.

- **Parquet** is another columnar data format. It was created by Cloudera and Twitter. A Parquet file contains groups of rows. The data for each column is stored together in the same row group. Each row group contains one or more blocks of data. A Parquet file contains metadata describing the set of rows in each block. An application can use this metadata to quickly locate the appropriate block for a given set of rows and retrieve the data in the columns specified for those rows. Parquet specializes in the efficient storage and processing of nested data types. It supports highly efficient compression and encoding schemes.

- **SequenceFiles** : Serialized data storage in Hadoop
  - SequenceFiles is a binary file format used in Hadoop to store serialised data. It offers several advantages over plain text formats such as CSV:
  - Efficiency: SequenceFiles compresses data, which reduces the storage space required and speeds up data reading and writing.
  - Flexibility: SequenceFiles can store data of various types, including primitive types (integers, floats, etc.), strings, arrays and complex objects.
  - Scalability: SequenceFiles is designed to run efficiently on large datasets distributed across multiple Hadoop nodes.

- Functionality:
  - Serialization: Data is serialized into a compact binary format before being stored in the file.
  - Records and keys: Data is organized into sequences of records, with each record having a unique key.
  - Indexing: SequenceFiles can be indexed, allowing quick access to data by key.

- Uses:
  - Structured data storage: SequenceFiles is ideal for storing structured data from databases, CSV files or other sources.
  - MapReduce output: SequenceFiles is a common output format for MapReduce jobs.
  - Intermediate for Hive and Pig: SequenceFiles can be used as an intermediate format for Hive and Pig processing.
- Benefits:
  - Efficiency: Saves storage space and speeds up data access.
  - Flexibility: Storage of various types of data.
  - Scalability: Efficient operation on large datasets.
- Disadvantages:
  - Complexity: More complex to use than plain text formats.
  - Hadoop dependency: Requires Hadoop to read and write files.

Annex : RDBM port

SGBD	Port par défaut	Protocole
MySQL	3306	TCP/IP
PostgreSQL	5432	TCP/IP
Oracle	1521	TCP/IP
Microsoft SQL Server	1433	TCP/IP
SQLite	Pas de port par défaut	Fichier
MongoDB	27017	TCP/IP
Cassandra	9042	TCP/IP
HBase	9090	TCP/IP

IBM Db2: 50000
Sybase ASE: 5000
SAP ASE: 1433
Informix: 1521
Teradata: 1025

Annex : Coudera certification

Cloudera Certified Professional (CCP)

The industry's most demanding performance-based certifications, CCP evaluates and recognizes a candidate's mastery of the technical skills most sought after by employers.

Certification overview	Description
CCP Data Engineer	CCP Data Engineers possesses the skills to develop reliable, autonomous, scalable data pipelines that result in optimized data sets for a variety of workloads.

Cloudera Certified Associate (CCA)

CCA exams test foundational skills and sets forth the groundwork for a candidate to achieve mastery under the CCP program

Certification overview	Description
CCA Spark and Hadoop Developer	A CCA Spark and Hadoop Developer has proven their core skills to ingest, transform, and process data using Apache Spark™ and core Cloudera Enterprise tools.
CCA Data Analyst	A CCA Data Analyst has proven their core analyst skills to load, transform, and model Hadoop data in order to define relationships and extract meaningful results from the raw input.
CCA Administrator	Individuals who earn the CCA Administrator certification have demonstrated the core systems and cluster administrator skills sought by companies and organizations deploying Cloudera in the enterprise.
CCA HDP Administrator Exam	The HDP Certified Administrator (HDPCA) exam has five main categories of tasks that involve: Installation, Configuration, Troubleshooting, High Availability and Security.